

PROJECT-4

A.

```
GNU nano 8.7.1 wait_example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;

    printf("Parent process started. PID = %d\n", getpid());

    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i, getpid(), getppid());

            sleep(i);

            printf("Child %d completed.\n", i);
            exit(0);
        }
    }

    printf("Parent waiting for all children using wait()...\n");

    for (i = 0; i < 3; i++)
    {
        wait(NULL);
    }

    printf("All children completed. Parent exiting.\n");

    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^_ Go To Line

```

administrator@VBCO-LAP-29:~/OSLab/OS-SKILL-1$ cd ~/OSLab
mkdir practical-4
cd practical-4
pwd
/home/administrator/OSLab/practical-4
administrator@VBCO-LAP-29:~/OSLab/practical-4$ nano wait_example.c
administrator@VBCO-LAP-29:~/OSLab/practical-4$ ls
gcc wait_example.c -o wait_example
ls
./wait_example
ls -l
wait_example.c
wait_example wait_example.c
Parent process started. PID = 15776
Child 1: PID = 15777, Parent PID = 15776
Child 2: PID = 15778, Parent PID = 15776
Parent waiting for all children using wait()...
Child 3: PID = 15779, Parent PID = 15776
Child 1 completed.
Child 2 completed.
Child 3 completed.
All children completed. Parent exiting.
total 20
-rwxr-xr-x 1 administrator administrator 16296 Aug 16 08:50 wait_example
-rw-r--r-- 1 administrator administrator 787 Aug 16 08:50 wait_example.c
administrator@VBCO-LAP-29:~/OSLab/practical-4$

```

B.

```

administrator@VBCO-LAP-29:~/OSLab/practical-4$ nano waitpid_example.c
administrator@VBCO-LAP-29:~/OSLab/practical-4$ ls
gcc waitpid_example.c -o waitpid_example
ls
./waitpid_example
wait_example wait_example.c waitpid_example.c
wait_example wait_example.c waitpid_example waitpid_example.c
Parent process started. PID = 15847
Child 1: PID = 15848, Parent PID = 15847
Child 2: PID = 15849, Parent PID = 15847
Parent waiting for specific children using waitpid()...
Child 3: PID = 15850, Parent PID = 15847
Child 3 completed.
Child 2 completed.
Child 1 completed.
Parent collected Child 1 (PID = 15848)
Parent collected Child 2 (PID = 15849)
Parent collected Child 3 (PID = 15850)
All children completed. Parent exiting.
administrator@VBCO-LAP-29:~/OSLab/practical-4$ |

```

```
GNU nano 8.7.1 waitpid_example.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;
    pid_t children[3];

    printf("Parent process started. PID = %d\n", getpid());

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(3 - i);

            printf("Child %d completed.\n", i + 1);
            exit(0);
        }

        children[i] = pid;
    }

    printf("Parent waiting for specific children using waitpid()...\n");

    for (i = 0; i < 3; i++)
    {
        waitpid(children[i], NULL, 0);

        printf("Parent collected Child %d (PID = %d)\n",
            i + 1, children[i]);
    }

    printf("All children completed. Parent exiting.\n");

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^A Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

C.

```
GNU nano 8.7.1 zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping without calling wait()...\n");
        sleep(30);

        printf("Parent exiting.\n");
    }

    return 0;
}
```

```

administrator@VBCO-LAP-29:~$ nano zombie.c
administrator@VBCO-LAP-29:~$ ls
gcc zombie.c -o zombie
ls
./zombie
OS-skill-1  oslab          practical2.c      pro1.c
OSLab       practical1     practical_01.c.save  program1.m.save
Practical2  practical1.c   pro1              zombie.c
OS-skill-1  practical1     pro1              zombie.c
OSLab       practical1.c   pro1.c
Practical2  practical2.c   program1.m.save
oslab       practical_01.c.save  zombie
Parent process: PID = 522
Child PID = 523
Parent sleeping without calling wait()...
Child process: PID = 523
Child is terminating...
Parent exiting.
administrator@VBCO-LAP-29:~$ |

```

D.

```

administrator@VBCO-LAP-29:~/OSLab/practical-4$ pwd
./zombie &
ps -o pid,ppid,state,cmd --forest
ps -ef | grep '\[zombie\]'
pstree -p
ps -o pid,ppid,state,cmd --forest
/home/administrator/OSLab/practical-4
[1] 603
Parent process: PID = 603
Child PID = 606
Parent sleeping without calling wait()...
Child process: PID = 606
Child is terminating...
  PID  PPID  S  CMD
  304   302  S  -bash
  603   304  S   \_ ./zombie
  606   603  Z   |   \_ [zombie] <defunct>
  610   304  R   \_ ps -o pid,ppid,state,cmd --forest
adminis+ 606   603  0 09:08 pts/0    00:00:00 [zombie] <defunct>
systemd(1)─agetty(196)
            └─chronyd-starter(110)─chronyd(228)─chronyd(239)
            └─cron(111)
            └─dbus-daemon(112)
            └─init-systemd(Ub(2))─SessionLeader(301)─Relay(30+)
                                   │   │   │   │   │
                                   │   │   │   │   └─{init}(6)─{init}(7)
                                   │   │   │   │   └─login(305)─bash(401)
                                   │   │   │   │   └─{init-systemd(Ub)}(8)
                                   │   │   │   └─networkd-dispat(130)
                                   │   │   └─rsyslogd(181)─{rsyslogd}(203)
                                   │   │                   │   │   └─{rsyslogd}(204)
                                   │   │                   │   │   └─{rsyslogd}(205)
                                   │   │   └─systemd(369)─(sd-pam)(371)
                                   │   └─systemd-journal(49)
                                   │   └─systemd-logind(160)
                                   │   └─systemd-resolve(81)
                                   │   └─systemd-udevd(88)─(udev-worker)(591)
                                                           └─(udev-worker)(592)
                                   └─unattended-upgr(236)─{unattended-upgr}(282)
                                   └─wsl-pro-service(162)─{wsl-pro-service}(221)
                                                           │   │   │   │   │
                                                           │   │   │   │   └─{wsl-pro-service}(222)
                                                           │   │   │   │   └─{wsl-pro-service}(223)
                                                           │   │   │   │   └─{wsl-pro-service}(224)
                                                           │   │   │   │   └─{wsl-pro-service}(227)
                                                           │   │   │   │   └─{wsl-pro-service}(229)
                                                           │   │   │   │   └─{wsl-pro-service}(230)
                                                           │   │   └─PID  PPID  S  CMD
                                                           │   │       304   302  S  -bash
                                                           │   │       603   304  S   \_ ./zombie
                                                           │   │       606   603  Z   |   \_ [zombie] <defunct>
                                                           │   │       629   304  R   \_ ps -o pid,ppid,state,cmd --forest
administrator@VBCO-LAP-29:~/OSLab/practical-4$ |

```

E.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent waiting for child using wait()...\n");

        wait(NULL);

        printf("Child has been collected by the parent.\n");
        printf("No zombie process remains.\n");
        printf("Parent exiting.\n");
    }

    return 0;
}
```

```

administrator@VBCO-LAP-29:~/OSLab/practical-4$ nano zombie_fixed.c
administrator@VBCO-LAP-29:~/OSLab/practical-4$ ls
gcc zombie_fixed.c -o zombie_fixed
ls
./zombie_fixed
ps -o pid,ppid,state,cmd --forest
ps -o pid,ppid,state,cmd | grep zombie_fixed
ls -l
wait_example wait_example.c waitpid_example waitpid_example.c zombie zombie.c zombie_fixed.c
wait_example waitpid_example zombie zombie_fixed
wait_example.c waitpid_example.c zombie.c zombie_fixed.c
Parent process: PID = 754
Child PID = 755
Parent waiting for child using wait()...
Child process: PID = 755
Child is terminating...
Child has been collected by the parent.
No zombie process remains.
Parent exiting.
  PID  PPID S  CMD
  304   302 S  -bash
  761   304 R  \_ ps -o pid,ppid,state,cmd --forest
  768   304 S  grep --color=auto zombie_fixed
total 80
-rwxr-xr-x 1 administrator administrator 16296 Aug 16 08:50 wait_example
-rw-r--r-- 1 administrator administrator  787 Aug 16 08:50 wait_example.c
-rwxr-xr-x 1 administrator administrator 16352 Aug 16 08:56 waitpid_example
-rw-r--r-- 1 administrator administrator  970 Aug 16 08:55 waitpid_example.c
-rwxr-xr-x 1 administrator administrator 16208 Aug 16 09:02 zombie
-rw-r--r-- 1 administrator administrator  590 Aug 16 09:02 zombie.c
-rwxr-xr-x 1 administrator administrator 16208 Aug 16 09:13 zombie_fixed
-rw-r--r-- 1 administrator administrator  721 Aug 16 09:13 zombie_fixed.c
administrator@VBCO-LAP-29:~/OSLab/practical-4$

```